

Catalogo de figuras — TFG IDS-ML-Payload

Inaki Moreno — material visual para la memoria y la defensa

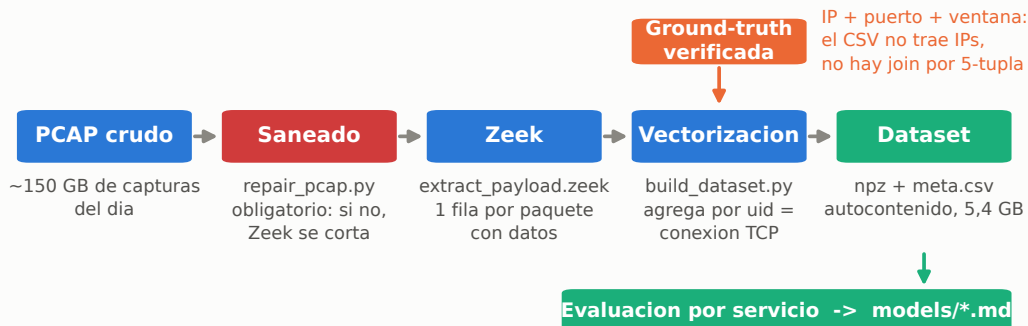
Generado por `scripts/figuras/catalogo_pdf.py`. Cada figura existe en `figuras/pdf/` (vectorial, para la memoria) y en `figuras/png/` (para la presentacion).

Bloque 0 — Diagramas conceptuales

F01_pipeline

De la captura de red al resultado

Cada día del dataset recorre esta cadena una sola vez. Los PCAP y los TSV intermedios se borran después (son re-derivables); lo que se conserva es el .npz vectorizado, que basta para re-etiquetar y re-evaluar sin volver a descargar nada.

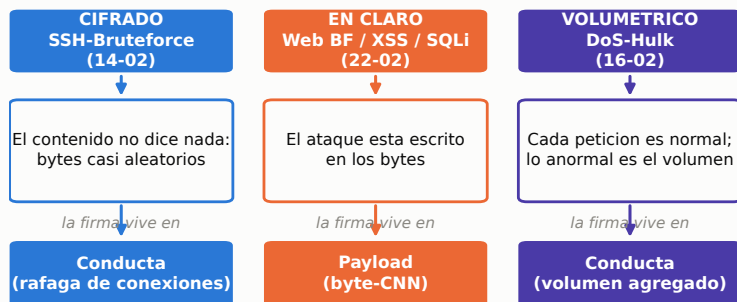


Scripts en `scripts/zeek/`. El saneado previo y el etiquetado por ground-truth verificada no son detalles de implementación: cada uno corrige un error que había invalidado resultados (Hallazgos 2 y 3).

F02_mapa_regimenes

Los tres regimenes de firma

El trabajo se estructura sobre tres ataques deliberadamente opuestos. Cada uno esconde su firma en un sitio distinto, y ese es el argumento: una sola vista de los datos es suficiente para uno de ellos y ciega para otro.



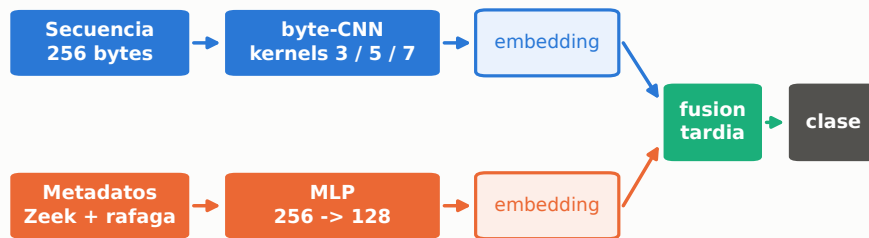
Ninguna vista gana en los tres -> la deteccion robusta exige combinar payload y conducta

Los tres días proceden del mismo dataset (CSE-CIC-IDS2018) y se evalúan con idéntico protocolo, de modo que las diferencias entre ellos son atribuibles al ataque y no al montaje experimental.

F03_arquitectura

El híbrido de dos ramas con fusion tardia

Cada vista se procesa con la arquitectura que le corresponde —convolucion para la secuencia de bytes, perceptron para los escalares de flujo— y solo se combinan al final, cuando ambas son ya representaciones de dimension comparable.



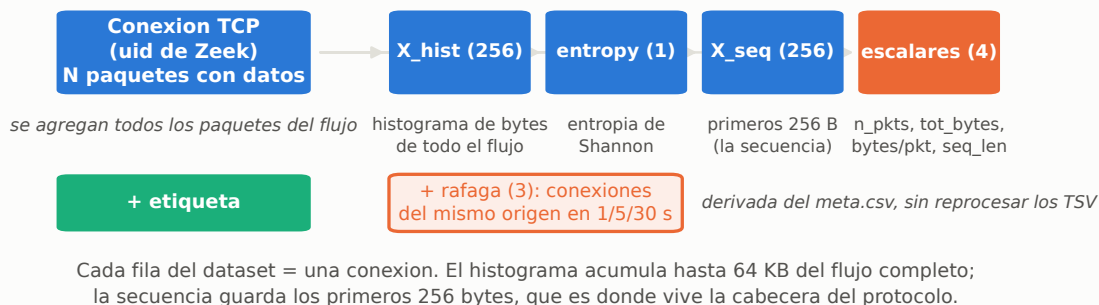
La fusion INGENUA concatenaria aqui las 257 dimensiones crudas del payload con los 7 escalares de Zeek: los escalares utiles quedarian diluidos. Entrenar una rama por vista y fusionar los embeddings lo evita.

Implementado en PyTorch (CPU): scripts/zeek/dl_models.py y train_dl_hybrid.py. Resultados en F23 y en models/phase2_dlhybrid_*.md.

F04_anatomia_flujo

Anatomia de un ejemplo del dataset

La unidad de analisis es la conexion TCP identificada por el uid de Zeek, no el paquete individual. De cada conexion se derivan cuatro representaciones distintas del mismo trafico, y son esas vistas las que se comparan a lo largo del trabajo.

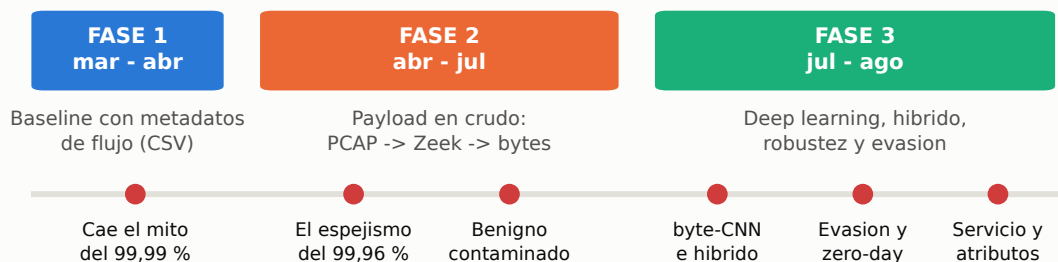


Construido por scripts/zeek/build_dataset.py (clase FlowAcc). Confirmado como correcto en la revision del tutor del 11 de agosto.

F05_cronologia

Cronologia: el proyecto avanza corrigiendose

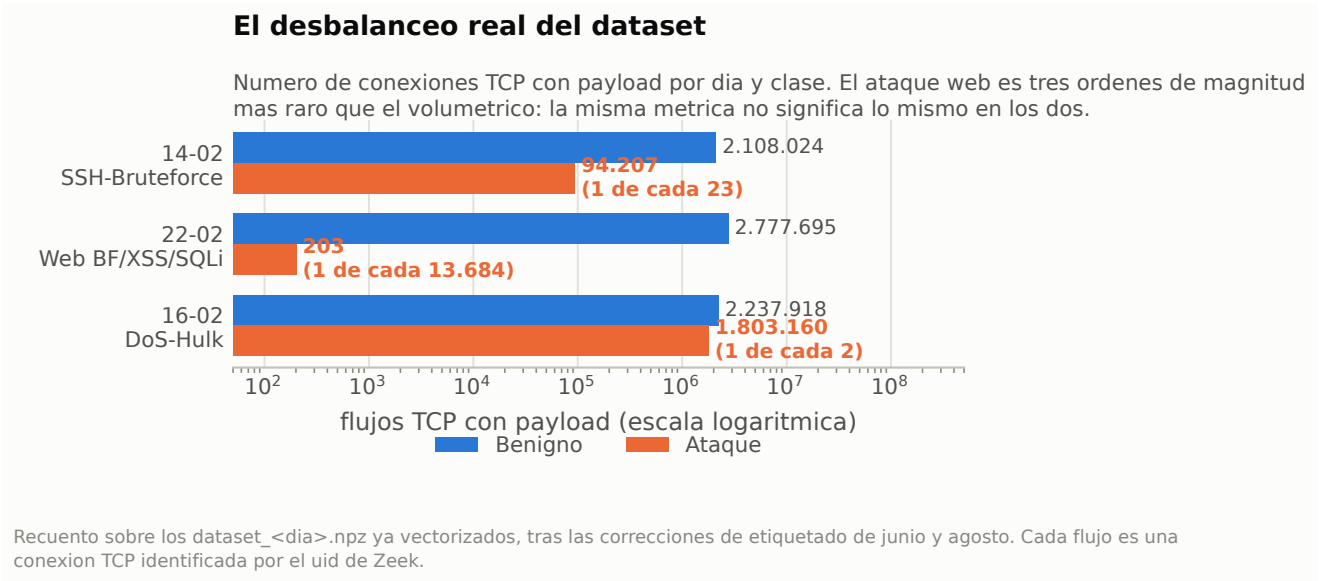
Los seis hitos marcados son los momentos en que una medida que parecia buena resultado ser un artefacto y hubo que rehacer el analisis. Es la linea argumental de la memoria.



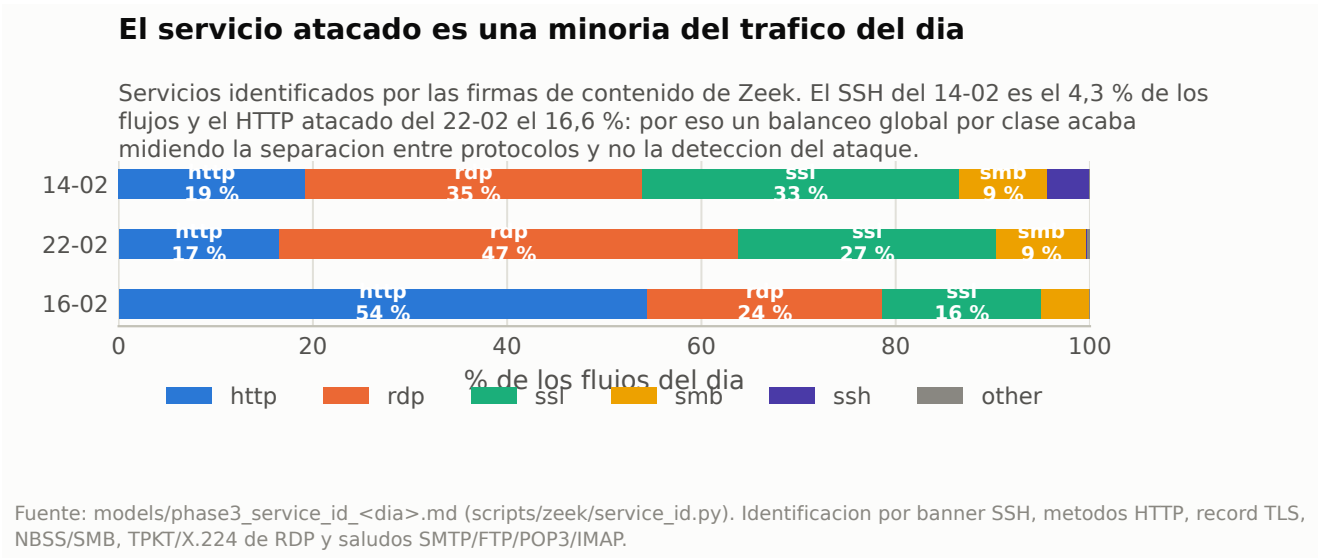
Diario completo y fechado en README.md; resumen ejecutivo en SUMMARY.md.

Bloque 1 — Los datos

F06_composicion_clases



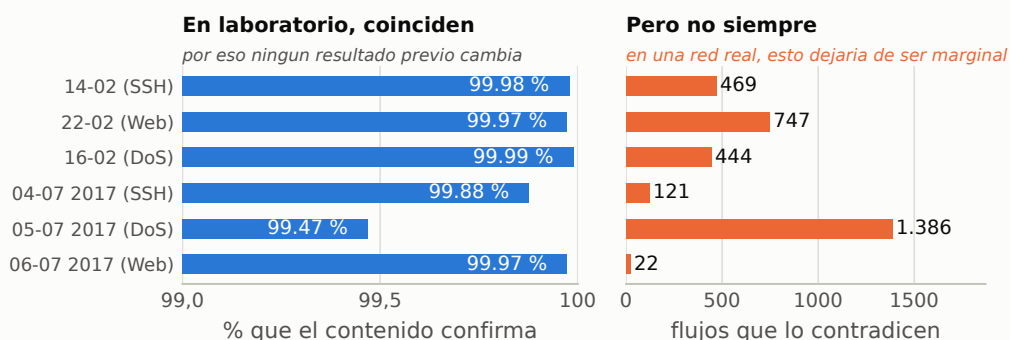
F07_servicios_por_dia



F08_servicio_vs_puerto

El servicio se identifica por el contenido, no por el puerto

El puerto, como la IP, depende de la configuración de cada red: no es un atributo independiente del contexto. Un modelo que lo use no generaliza a una red donde los servicios escuchen en otros puertos.

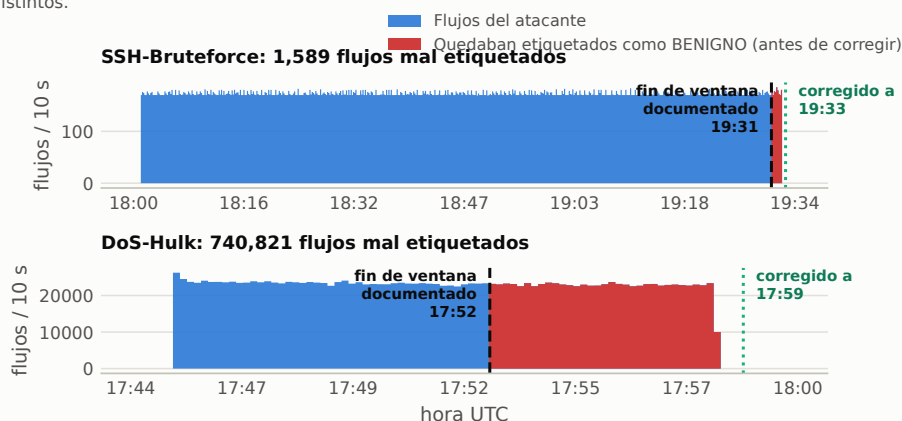


Fuente: models/phase3_service_id_<dia>.md. Ejemplos concretos del desacuerdo: 99 flujos HTTP fuera del puerto 80 el 22-02, y 75 flujos del puerto 22 que no son SSH el 14-02 (que salen del conjunto benigno: 433 -> 358).

F09_error_ventana

El atacante seguía actuando después del fin de ventana documentado

Actividad del atacante sobre el servicio atacado, en tramos de 10 s. Todo lo que queda a la derecha del corte se etiquetaba como tráfico legítimo, contaminando el conjunto benigno con el propio ataque. Ocurrió en dos días distintos.



Verificado sobre el dataset completo (no sobre una sola captura): la actividad del atacante es continua, con gap máximo de 1 s en SSH y 0,2 s en DoS. Corregido en attack_metadata.py y re-etiquetado con relabel_dataset.py sin reprocesar los PCAP.

F10_volumen_procesado

Escala del trabajo experimental

6

días procesados

3 de CSE-CIC-IDS2018 + 3 de saneadas y procesadas con CIC-IDS2017 Zeek en Docker

~150 GB

de capturas PCAP

9,4 M

flujos TCP vectorizados

histograma, entropía y secuencia por conexión

3

regímenes de firma

cifrado, en claro y volumétrico

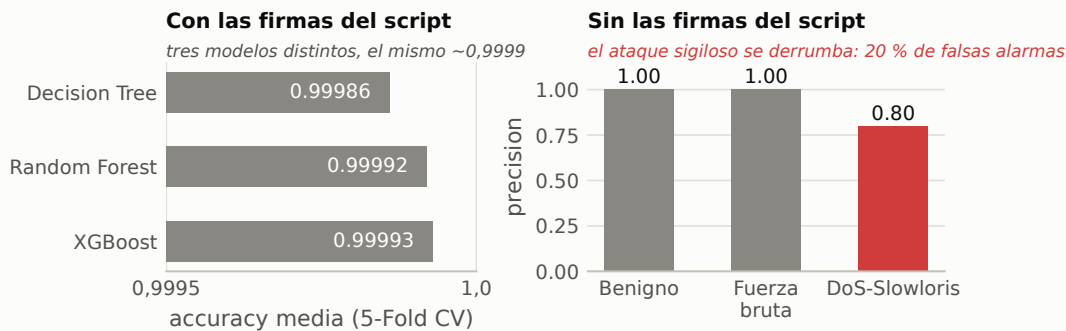
Los PCAP y TSV intermedios se borran tras vectorizar cada día (son re-derivables desde S3); se conservan los dataset_<dia>.npz autocontenidos, 5,4 GB en total.

Bloques 2 y 3 — Honestidad metodológica

F11_fase1_robustez

Fase 1: cae el mito del 99,99 %

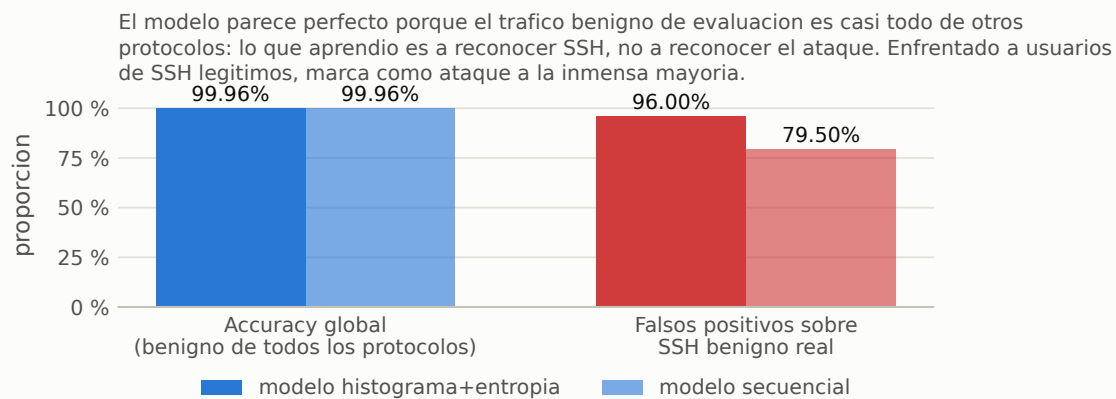
Al eliminar las variables que memorizan el tamaño exacto de los paquetes de la herramienta de ataque y balancear las clases, el DoS-Slowloris (de baja tasa) deja de distinguirse del tráfico lento legítimo.



Izquierda: 5-Fold CV multiclase sin balancear. Derecha: XGBoost con undersampling y sin Init_Win_bytes / Fwd Seg Size Min. Fuente: notebooks/02_multiclass_benchmark.ipynb, README del 27 de abril.

F12_espejismo

El 99,96 % era un espejismo del desbalanceo de servicios

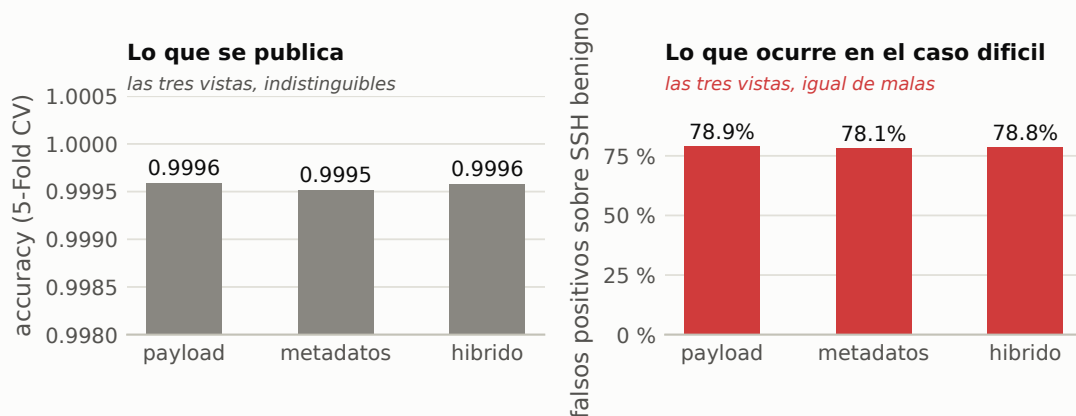


Accuracy: test held-out 20 % sobre el dataset balanceado (185.236 muestras). Falsos positivos: auditoría sobre los 2.022 flujos benignos reales del puerto 22. Fuentes: models/phase2_results.md, scripts/zeek/honest_check.py.

F13_balanceo_global

Añadir metadatos 'a ciegas' no arregla nada

Con balanceo global por clase, payload, metadatos e híbrido son estadísticamente equivalentes: las tres aprenden el mismo atajo ('SSH = ataque'). El espejismo es estructural al protocolo de evaluación, no exclusivo del payload.

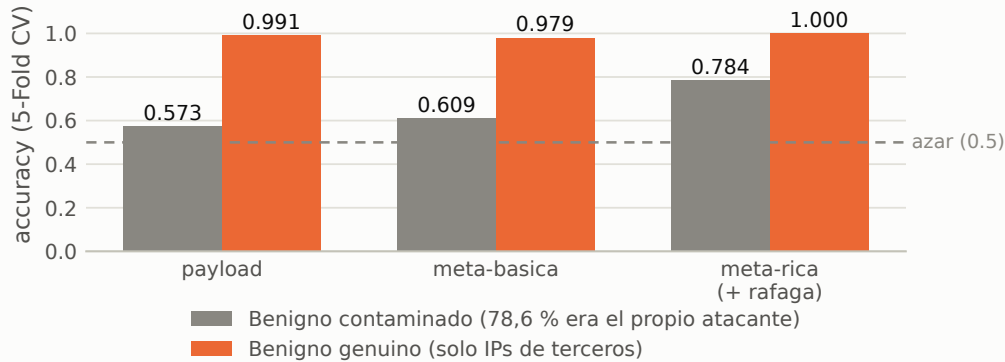


Mismos flujos y mismo protocolo en las tres vistas (MLP 256->128, undersampling global, 5-Fold CV + test held-out). Recall del ataque = 100 % en las tres. Fuente: models/phase2_hybrid_compare.md.

F14_benigno_contaminado

El 'techo' de los metadatos era un error de etiquetado

Comparadas contra el benigno contaminado, las tres vistas rozan el azar y parece que el problema es la vista. Con el benigno genuino, las tres separan— y la conductual llega a 1,000.



Balanceo 1:1 dentro del servicio SSH, MLP 256->128, 5-Fold CV + test held-out. El benigno contaminado son 2.022 flujos del puerto 22, de los que 1.589 eran del atacante 13.58.98.64 fuera de ventana. Fuente: models/phase2_by_service_rich.md.

F15_composicion_benigno

El mismo error de etiquetado, dos veces

Composicion real del conjunto 'benigno' del servicio atacado antes de corregir la ventana. La ventana documentada terminaba antes de que el atacante dejara de actuar, asi que su propia cola quedaba etiquetada como trafico legitimo.



Verificado sobre el dataset completo: en ambos casos la actividad del atacante es continua (gap maximo 1 s en SSH, 0,2 s en DoS) y se extiende mas alla del fin de ventana documentado. Corregido en scripts/zeek/attack_metadata.py y re-etiquetado con relabel_dataset.py.

Bloque 4 — Los tres regimenes (nucleo de la tesis)

F16_matriz_regimen_vista

Ninguna vista gana en los tres regimenes

Acierto por regimen de ataque y vista de datos. 'Evadible' marca las celdas cuyo acierto se hunde al camuflar los primeros bytes (Hallazgo 11).

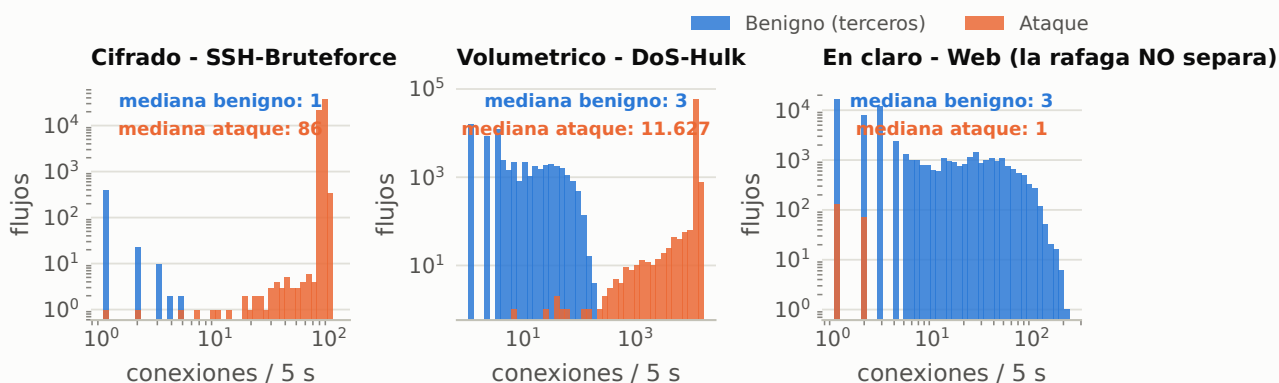
Cifrado SSH-Bruteforce	0.991 <i>evadible</i>	0.979	1.000	1.000
En claro Web BF/XSS/SQLi	0.988	0.929	1.000	0.998
Volumetrico DoS-Hulk	1.000 <i>evadible</i>	0.971	1.000	<i>no medido</i>
	Payload (bytes)	Metadatos per-flujo	Conducta (rafaga)	Hibrido (2 ramas)

Accuracy CV / macro-F1 out-of-fold, 5-Fold estratificado, balanceo 1:1, benigno genuino (sin flujos del atacante). Fuentes: models/phase2_by_service_rich.md, phase2_by_service_web.md, phase2_dos_burst_*.md, phase2_dlhybrid_*.md, phase2_rigor_*.md.

F17_rafaga_distribucion

La rafaga separa dos regimenes sin mirar un solo byte — y falla en el tercero

Distribucion del nº de conexiones por origen en ventanas causales de 5 s (ambos ejes en escala logaritmica). En el ataque web, de bajo volumen, la conducta es ciega: es justo donde el payload gana.

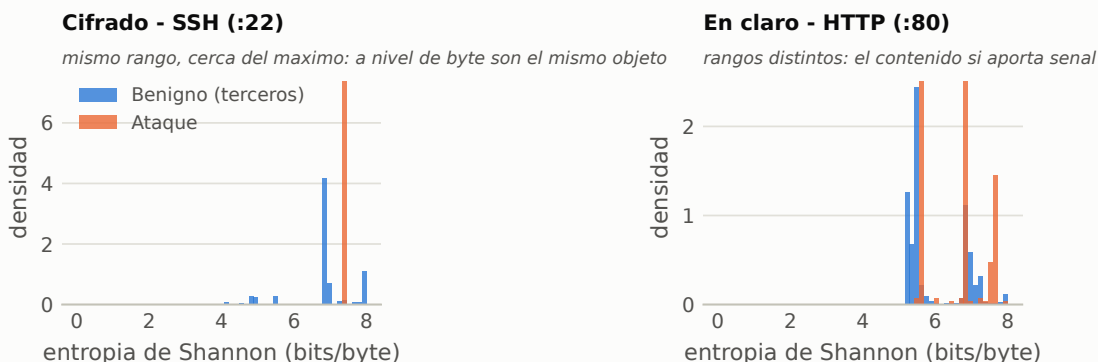


Calculado sobre el dia completo (todos los origenes) y segmentado despues por clase, igual que en scripts/zeek/honest_by_service_rich.py. Benigno = solo IPs de terceros.

F18_entropia_distribucion

El cifrado borra la senal del payload; el texto en claro la conserva

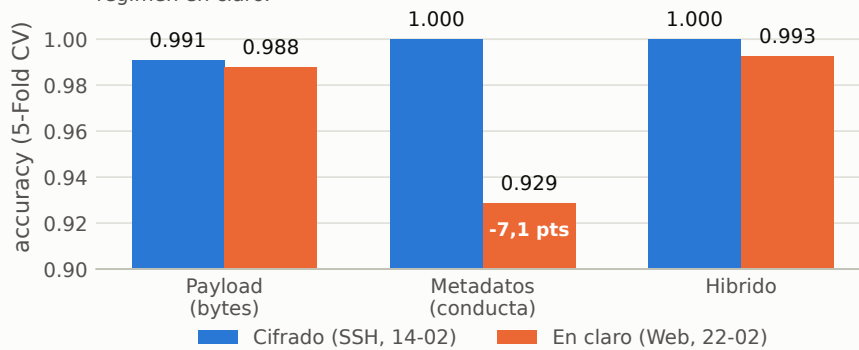
Entropia por flujo dentro del servicio atacado. El maximo teorico es 8 bits/byte: cuanto mas cerca, mas indistinguible del ruido.



Entropia de Shannon calculada por scripts/zeek/build_dataset.py sobre el payload agregado de cada conexion TCP (hasta 64 KB). Benigno = solo IPs de terceros.

Cada vista gana en su regimen: el resultado se invierte

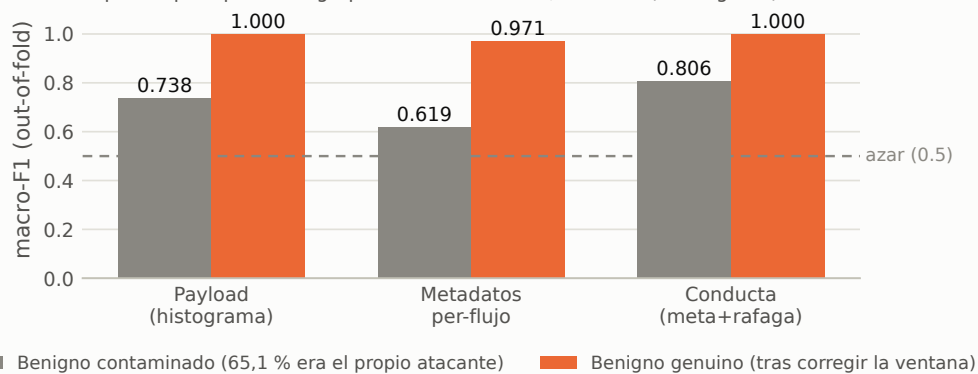
La misma comparacion de vistas sobre dos ataques opuestos, con identico protocolo de evaluacion. Lo que cambia no es que el payload suba, sino que los metadatos per-flujo se desploman en el regimen en claro.



Accuracy 5-Fold CV, balanceo 1:1 dentro del servicio atacado, benigno genuino. SSH: models/phase2_by_service_rich.md (escenario limpio) y phase2_dlhybrid_Wednesday-14-02-2018_binary.md. Web: models/phase2_by_service_web.md.

El 'per-flujo no sirve' del DoS era un artefacto del etiquetado

Comparar ataque contra ataque hundia las tres vistas. Con benigno real, el payload tambien separa— pero por un fingerprint de herramienta, evadible (Hallazgo 11).

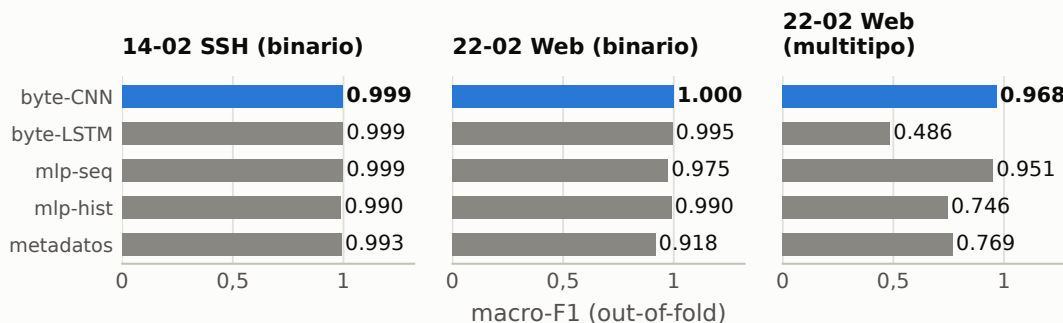


Mismo protocolo en ambas tandas: balanceo 1:1, --cap 4000, out-of-fold 5-Fold, seleccion por servicio. Fuente: models/phase2_dos_burst_Friday-16-02-2018.md (15-jul vs 11-ago).

Bloque 5 — Deep learning

El byte-CNN es el modelo mas fuerte sobre la secuencia de bytes

La convolucion multi-kernel aprende n-gramas de bytes (los tokens del ataque). En binario casi todo empata; la diferencia aparece al identificar el TIPO de ataque, donde el LSTM se hunde y los baselines shallow se quedan atras.

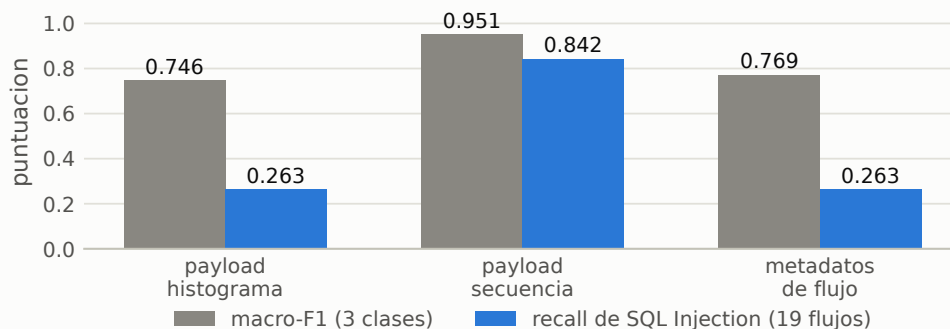


Predicciones out-of-fold (5-Fold estratificado) sobre la misma particion para los cinco modelos. PyTorch en CPU. Fuente: models/phase2_dl_<dia>_<area>.md (scripts/zeek/train_dl_payload.py). El dia DoS se excluye: sus cifras del 15-jul quedaron invalidadas por el re-etiquetado del 11-ago.

F22_multitipo_web

El tipo de ataque vive en la secuencia, no en el histograma

Clasificacion de los 203 flujos de ataque web en sus tres tipos. El histograma y los metadatos aciertan el tipo mayoritario pero fallan la inyeccion SQL: solo la secuencia de bytes conserva los tokens que la identifican.

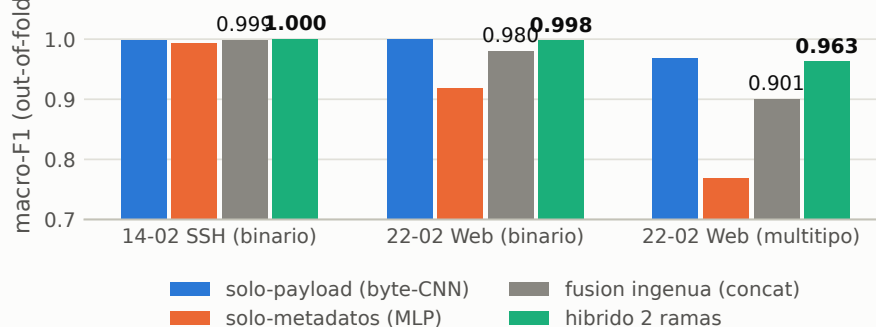


Predicciones out-of-fold (5-Fold estratificado). Clases: 142 Brute Force -Web, 42 XSS, 19 SQL Injection. Fuente: models/phase2_multitype_web.md (scripts/zeek/multitype_web.py).

F23_hibrido_ramas

La fusion bien hecha es robusta en los dos regimenes

Cada vista suelta gana en su regimen y pierde en el otro. La fusion ingenua (concatenar features crudas) queda por debajo en las tres tareas; el hibrido de dos ramas con fusion tardia se mantiene arriba en todas.

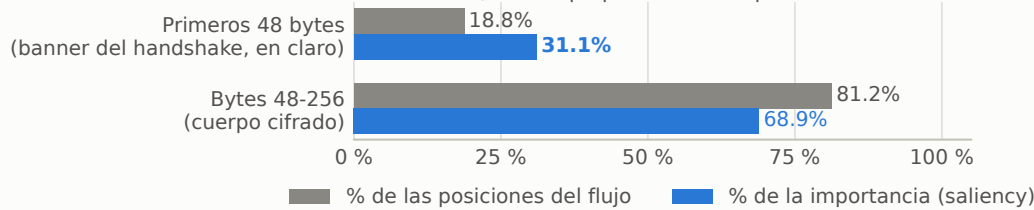


Predicciones out-of-fold (5-Fold estratificado). El hibrido entrena una rama CNN sobre el payload y una MLP sobre los metadatos, y fusiona los embeddings antes del clasificador. Fuente: models/phase2_dlhibrido_*.md (scripts/zeek/train_dl_hybrid.py).

F24_saliency

El modelo no lee el cifrado: lee la cabecera en claro

El 18,8 % inicial del flujo concentra el 31,1 % de la importancia. Es la zona del banner SSH, que viaja en claro antes de negociar el cifrado: la señal es real, pero es la huella del cliente del atacante, no una propiedad del ataque.



Saliency = |gradiente| de la salida respecto a la entrada, promediado sobre los flujos de ataque. Entropía media de ambas clases ~7,4 bits/byte (indistinguibles a nivel de contenido). Fuente: models/phase2_saliency_ssh.md (scripts/zeek/eval_saliency.py).

F25_banners

La 'senal' del payload en SSH es un unico cliente

Primeros 42 bytes de conexiones reales del dataset (los no imprimibles se muestran como ·). El atacante usa siempre paramiko; el trafico legitimo trae PuTTY, libssh2, OpenSSH... Falsear el banner anula la deteccion, y eso es exactamente lo que mide el Hallazgo 11.

Flujos del ATACANTE

```
SSH-2.0-paramiko_2.0.0 SSH-2.0-OpenSSH_7.
SSH-2.0-OpenSSH_7.2p2 Ubuntu-4ubuntu2.4·S
```

(no hay mas: el atacante repite siempre el mismo cliente)

Flujos BENIGNOS (terceros)

```
·K·0·\s·h·G·^·/}WC·T
SSH-2.0-libssh2_1.7.0 SSH-2.0-OpenSSH_7.2
SSH-2.0-OpenSSH_7.2p2 Ubuntu-4ubuntu2.4·S
SSH-2.0-PuTTY·SSH-2.0-OpenSSH_7.2p2 Ubunt
{·e+N·fi·n·M·aB·EP=V·&·#·NL!·
$,8·#S·CDU·I·t·R·1·0l·B·G·}·
o\Q·d5·w·g·*,·9|·;·R;·
```

2 banners distintos en todo el ataque

81 banners distintos en el trafico legitimo

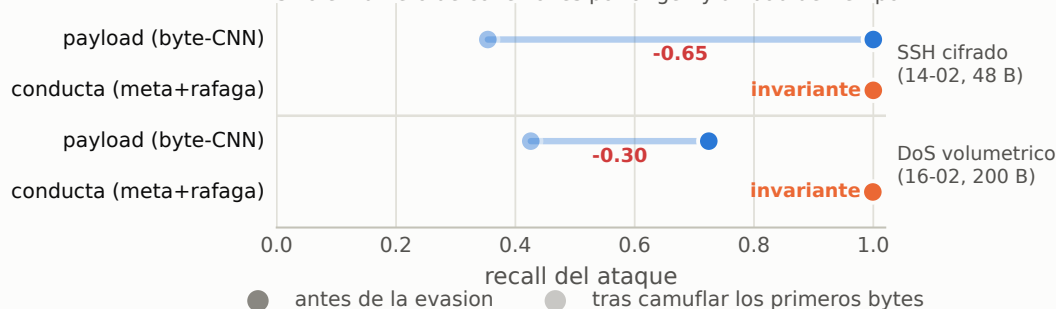
Muestras tomadas de dataset_Wednesday-14-02-2018.npz (X_seq, servicio SSH), benigno solo de IPs de terceros. Recuento de banners distintos: models/phase2_by_service_rich.md.

Bloque 6 — Robustez

F26_evasion

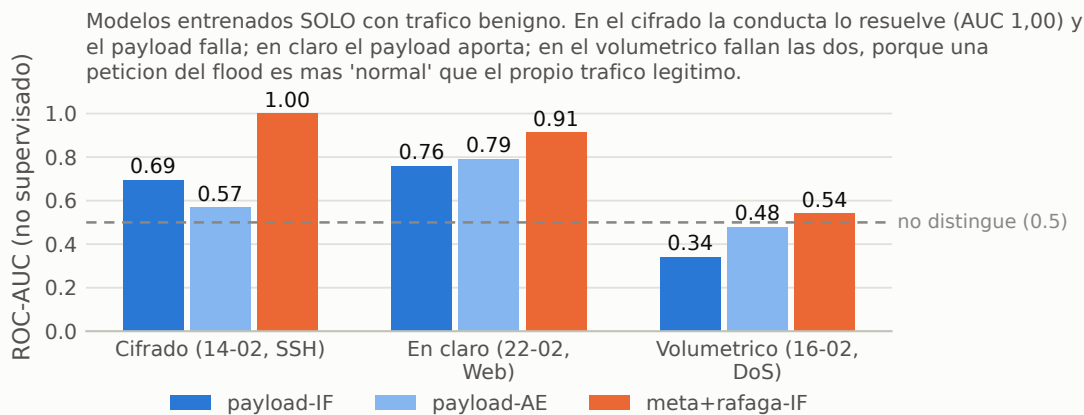
La firma de bytes se evade; la conductual no

El atacante copia los primeros bytes de un cliente legitimo y no cambia nada mas. El detector de payload pierde la mitad de los ataques; el conductual ni se entera, porque no mira el contenido sino el numero de conexiones por origen y unidad de tiempo.



Binario Attack vs Benign en el servicio atacado, balanceo 1:1, split 70/30. Se camuflan los primeros 48 B (SSH) / 200 B (DoS) de cada flujo de ataque. Fuente: models/phase2_evasion_*.md (scripts/zeek/evasion_test.py).

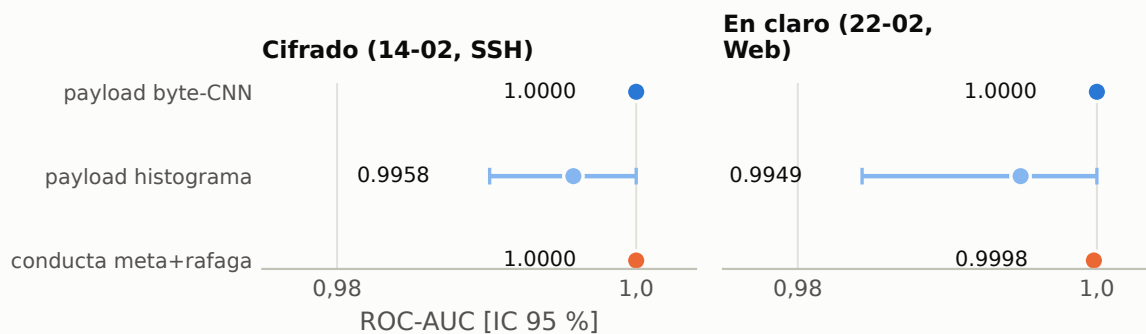
Sin etiquetas, la tesis se sostiene igual



IF = Isolation Forest, AE = autoencoder. Entrenados solo con benigno; el ataque nunca se ve en entrenamiento. Fuente: models/phase2_anomaly_*.md (scripts/zeek/anomaly_detection.py). El día DoS se midió antes del re-etiquetado del 11-ago.

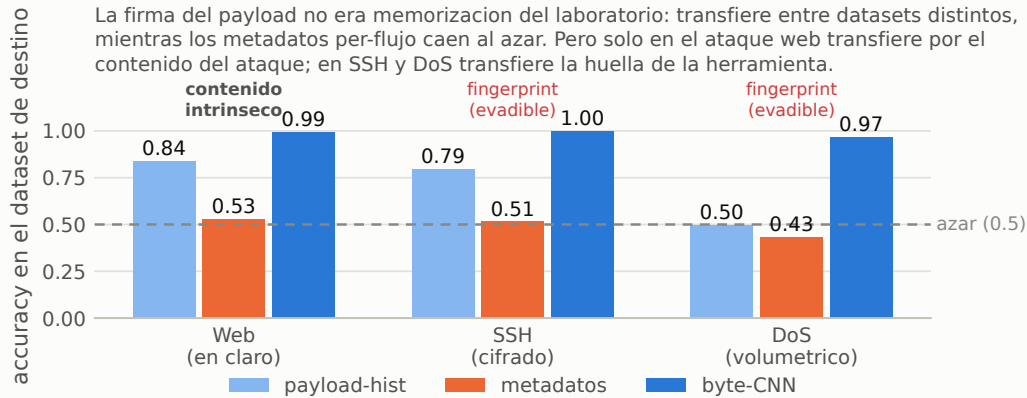
Las cifras, con su incertidumbre

ROC-AUC e intervalo de confianza al 95 % por bootstrap (2.000 remuestreos). Los intervalos son estrechos en los dos regimenes con N suficiente; el día DoS no aparece porque sus metricas de rigor se calcularon antes de corregir el etiquetado.



Out-of-fold 5-Fold, balanceo 1:1 (433 vs 433 en SSH; 203 vs 203 en web). Fuente: models/phase2_rigor_*.md (scripts/zeek/rigor_metrics.py), que ademas genera las curvas ROC y Precision-Recall completas en PNG.

El byte-CNN detecta en 2017 lo que aprendió en 2018

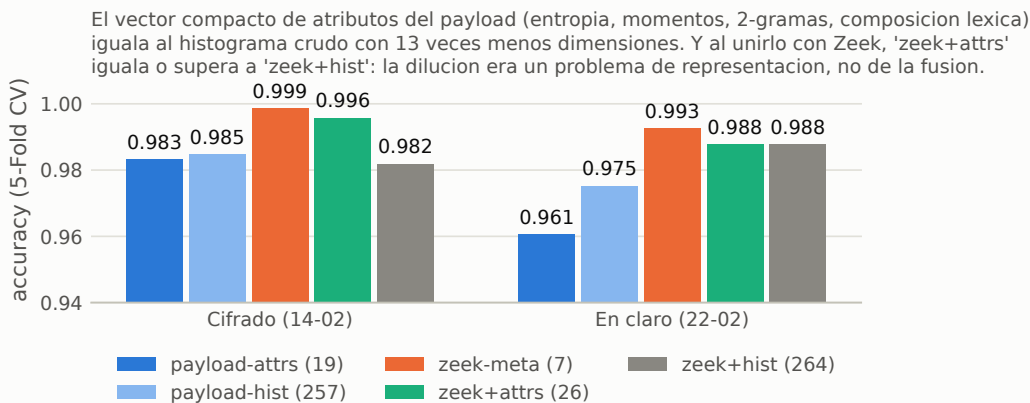


Entrenado en el día de 2018 y evaluado en el día analogo de 2017, balanceo 1:1 en ambos (0,5 = azar). Fuente: models/phase2_crossdataset_*.md (scripts/zeek/cross_dataset_eval.py). La dirección inversa (2017 -> 2018) se desploma por la menor diversidad del ataque de 2017.

Bloque 7 — Atributos del payload

F30_atributos_vistas

19 descriptores interpretables valen lo que 257 dimensiones

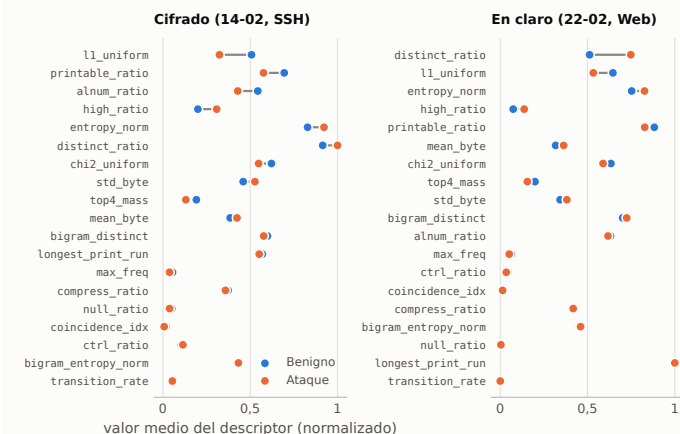


Mismo subconjunto en las cinco vistas: selección por servicio (DPI), balanceo 1:1, benigno limpio de la IP atacante, MLP 256->128, 5-Fold CV + test held-out. El día DoS se omite porque las cinco vistas saturan a 1,000 y no discriminan. Fuente: models/phase3_payload_attrs_*.md.

F31_atributos_separacion

Que mira cada regimen: los descriptores que separan no son los mismos

Media de cada descriptor por clase, ordenados por separacion. En el SSH cifrado manda la aleatoriedad (l1_uniform, entropia, riqueza de bytes); en el web en claro manda la composicion lexica del texto (distinct_ratio, l1_uniform, high_ratio).

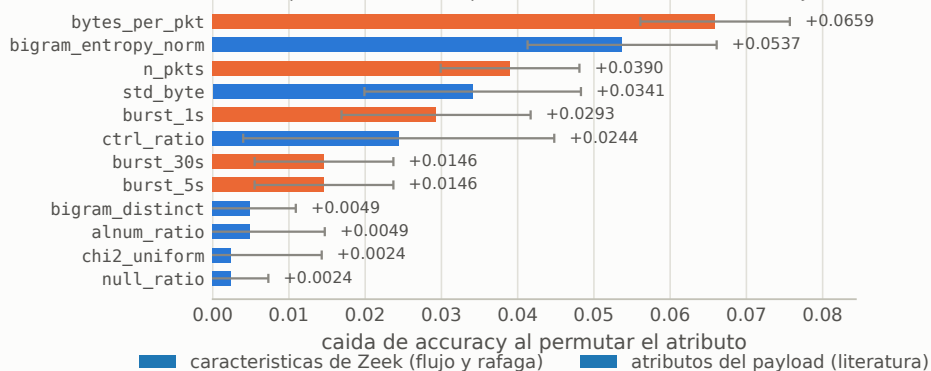


Descriptores normalizados a [0,1], calculados por scripts/zeek/payload_features.py sobre el payload agregado de cada conexon. Origen bibliografico de cada familia (PAYL, Anagram, Lyda & Hamrock) documentado en el modulo. Fuente: models/phase3_payload_attrs_*.md.

F32_importancia_permutacion

La union payload + Zeek usa de verdad las dos familias

Importancia por permutacion sobre la vista 'zeek+attrs' en el dia web. Las dos familias se alternan en la cabeza del ranking: ni el payload es un adorno sobre los metadatos ni al reves. Es la respuesta medida a la peticion del tutor de unir ambos conjuntos.



Caida media de accuracy en el test held-out sobre 5 repeticiones (barras de error = desviacion). Vista zeek+attrs = 7 caracteristicas de Zeek + 19 descriptores del payload. Fuente: models/phase3_payload_attrs_Thursday-22-02-2018_http.md.